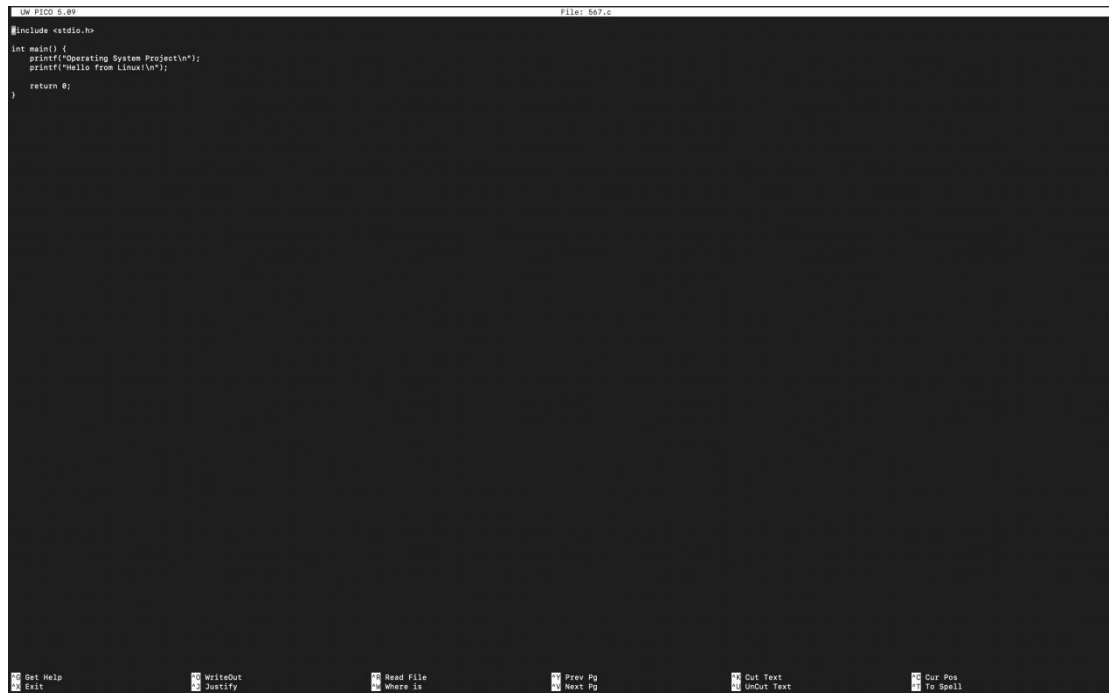


Week 2

SESSION-3

Task 1: To Install Linux VM, Configure GCC, Setup Git Repository, Create Project Structure, Understand Shell Architecture, Build Initial Make file.

CODE:

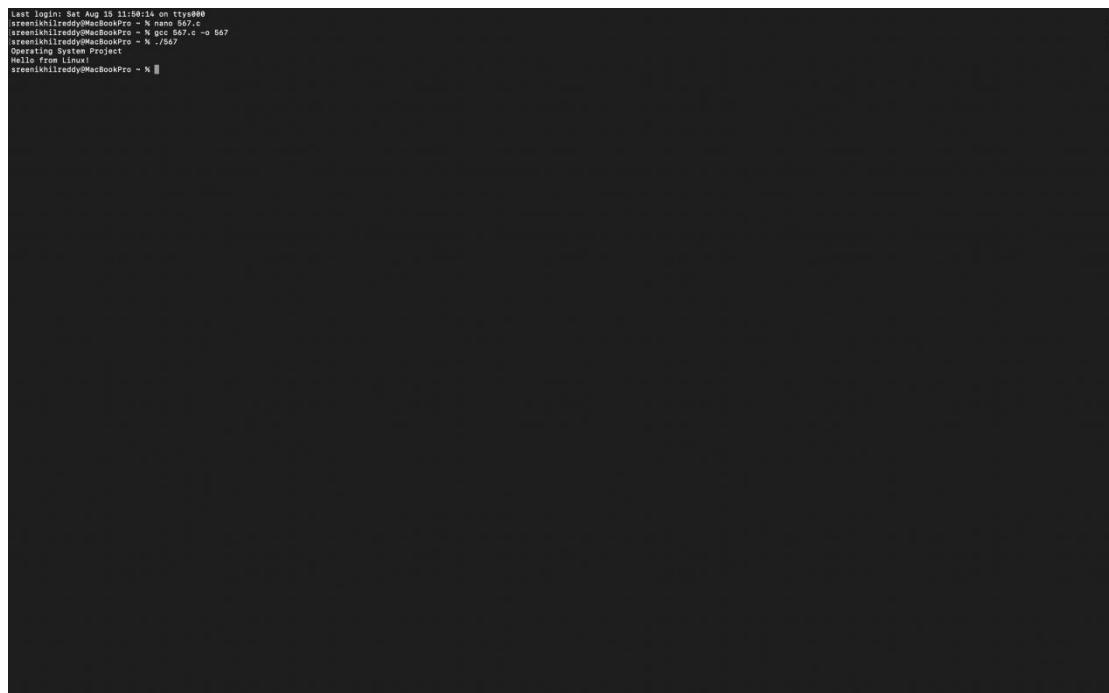


```
File: b07.c
#include <stdio.h>

int main() {
    printf("Operating System Project\n");
    printf("Hello from Linux\n");

    return 0;
}
```

OUTPUT:



```
Last login: Sat Aug 31 11:58:14 on tty000
screenkhilreddy@MacBookPro ~ % nano b07.c
screenkhilreddy@MacBookPro ~ % gcc b07.c -o b07
screenkhilreddy@MacBookPro ~ % ./b07
Operating System Project
Hello from Linux
screenkhilreddy@MacBookPro ~ %
```

SESSION-4

Task 1: To Split Input into Tokens, Identify Delimiters, Handle Whitespace, Create Token Structures, Validate Token Streams, Debug Parsing Output

CODE:.

```

1  UM FICD 3.89                               File: 456.c
2  #include <stdio.h>
3  #include <string.h>
4  #define MAX 50
5
6  typedef struct {
7      char value[MAX];
8  } token;
9
10 int main() {
11     char input[200];
12     token tokens[20];
13     int count = 0;
14
15     printf("Enter command: ");
16     fgets(input, sizeof(input), stdin);
17     input[strcspn(input, "\n")] = '\0';
18     char sp = strtok(input, " \\t");
19
20     while (sp != NULL && count < 20) {
21         strcpy(tokens[count].value, sp);
22         count++;
23         sp = strtok(NULL, " \\t");
24     }
25
26     printf("\nTokens:\n");
27     if (count == 0) {
28         printf("Invalid input\n");
29         return 1;
30     }
31     for (int i = 0; i < count; i++)
32         printf("Token %d: %s\n", i + 1, tokens[i].value);
33     printf("Token stream is valid.\n");
34     return 0;
35 }
36
37 Get Help                               WriteOut
38 Exit                                   Justify
39 Read File                             Prev Pg
40 Where Is                               Next Pg
41 Out Text                               Out Pos
42 UnOut Text                             To Spell

```

OUTPUT:

```

1  Last login: Sat Aug 15 12:21:44 on ttye000
2  streamahillreddy@MacBookPro ~ % nano 456.c
3  streamahillreddy@MacBookPro ~ % gcc 456.c -o 456
4  streamahillreddy@MacBookPro ~ % ./456
5  Enter command: ls
6
7  Tokens:
8  Token 1: [ls]
9  Token stream is valid.
10 streamahillreddy@MacBookPro ~ %

```

Task 2: To Design Parser Logic, Generate Parse Trees, Validate Syntax, Detect Errors, Handle Empty Commands, Produce Execution Structures.

```
UN FICD 5.89 File: 222.c
#include <stdio.h>
#include <string.h>

int main() {
    char input[100];
    char *cmd, *arg;

    printf("Enter command: ");
    fgets(input, sizeof(input), stdin);
    input[strcspn(input, "\n")] = '\0';

    // Handle empty command
    if (strlen(input) == 0) {
        printf("Error: Empty command\n");
        return 1;
    }

    // Parse command and argument
    cmd = strtok(input, " ");
    arg = strtok(NULL, " ");

    // Validate command
    if (cmd == NULL) {
        printf("Syntax Error\n");
        return 1;
    }

    // Generate parse tree
    printf("\nParse Tree:\n");
    printf("COMMAND: %s\n", cmd);

    if (arg != NULL)
        printf("    ARGUMENT: %s\n", arg);

    // Execution structure
    printf("\nExecution Structure:\n");
    printf("Command = %s\n", cmd);

    if (arg != NULL)
        printf("Argument = %s\n", arg);

    printf("\nSyntax is valid.\n");

    return 0;
}

Get Help      WriteOut    Read File    Prev Pg      Cut Text     Cur Pos
Exit          Justify      Where is     Next Pg      UnCut Text   To Spell

Last login: Sat Aug 15 12:23:27 on ttye000
screen@hilleddyMacBookPro ~ % nano 222.c

screen@hilleddyMacBookPro ~ % gcc 222.c -o 222
screen@hilleddyMacBookPro ~ % ./222
Enter command: ls

Parse Tree:
COMMAND: ls
Execution Structure:
Command = ls
Syntax is valid.
screen@hilleddyMacBookPro ~ %
```